```
Output:
object(stdClass)#1 (1) {
  ["beverage"]=>
  string(13) "White Russian"
}
```

`stdClass` is quite the workhorse. It is even possible to manipulate its properties at runtime. In the next example we use the object operator `->` to add a property to a `stdClass` object, and again to access the property's value:

```
$lebowski = new stdClass();
$lebowski->hates = 'The Eagles';
print "I really fn hate " . $lebowski->hates . ", man." . PHP_EOL;
```

```
Output:
I really fn hate The Eagles man.
```

It should be noted that runtime dynamic property creation is deprecated in PHP 8.2, although permitted for objects of type `stdClass`. Your own classes can use a new mechanism in PHP 8 called attributes to declare that they allow dynamic properties; we will cover attributes further on in the chapter. It is not permitted to add methods to an object. If Chapter 3, "Functions," made sense, you might be wondering if we could just add an anonymous function like this:

```
$lebowski->getMusicTaste = function () {
    echo "I really hate {$this->hates}, man."; // an object references
                                                itself via the reserved
                                                $this variable
};

var_dump($lebowski);
$lebowski->getMusicTaste();
```

```
Output:
object(stdClass)#1 (2) {
  ["hates"]=>
  string(10) "The Eagles"
  ["getMusicTaste"]=>
```